Experimento A, tercera parte: de sobrevivir a jugar con un propósito

Informe de avance · 3 de noviembre de 2026 · Cubre lo ocurrido desde el informe del 18 de octubre · Proyecto «Mundo simulado»

Pregunta de investigación. ¿Puede un personaje artificial en un mundo abierto (Minecraft) perseguir metas largas y útiles para una persona —«consígueme cinco diamantes», «construyeme este castillo»— combinando reflejos aprendidos, un planificador sin IA y, solo cuando hace falta, una IA grande que razona?

Lo que traía el informe 2. Un cuerpo que ya sobrevivía solo (16 de 16 noches) y una mente que planificaba por encima. **Lo que trae este:** el paso del «sobrevivir» al «jugar con un propósito»: roles de juego completos, un planificador barato (el *capataz*), un recolector que resuelve cadenas de requisitos, un catálogo de construcciones reales, y una forma de trabajar a dos personas, cada una con su agente, sin perder conocimiento.

~200	30	24	0
cambios de código (commits) en 16 días	decisiones razonadas y escritas	aprendizajes («esto falla por…»)	muertes en el banco del recolector

1. En pocas palabras

- **Se cambió de escala:** de habilidades sueltas (talar, refugiarse) a **roles de juego** de muchas horas —el jugador casual, el minero— que encadenan decenas de pasos.
- **Se separó el «qué toca ahora» de la IA cara:** un *capataz* escrito a mano decide la siguiente etapa mirando el inventario. No consume IA y es predecible.
- **Nace el recolector:** «consigue N diamantes» resuelve solo todo lo que hay en medio (pico, antorchas, bajar, túnel seguro, volver). Provisionalmente, **todos los materiales medidos pasan** su criterio y sin una sola muerte.
- **Los bots empiezan a construir obras reales** desde planos descargados, capa a capa, pidiendo materiales al cofre.
- **Método:** todo se mide con **bancos** repetibles cuyo criterio se fija *antes* de medir; el conocimiento se guarda en un **vault** compartido; y el coste en IA se vigila como una variable del experimento más.

> **Idea clave para entenderlo todo**
>
> En el informe 2 el personaje tenía **cuerpo** (reflejos rápidos) y **mente** (IA grande). Ahora tiene **tres capas**: entre ambas aparece el **capataz**, que hace de «jefe de obra»: no piensa en grande, pero sabe qué paso viene después. Así la IA cara solo se usa para lo que de verdad requiere razonar.

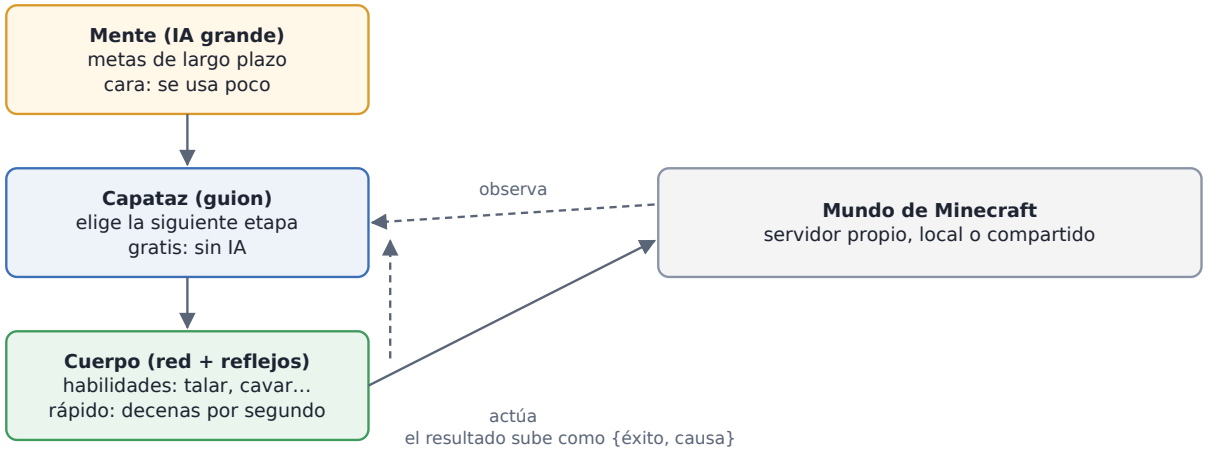


Figura 1. Las tres capas del personaje. Las órdenes bajan (mente → capataz → cuerpo) y los resultados suben como un par «éxito, causa». La flecha discontinua es la información que vuelve.

2. Cronología del periodo

El trabajo no fue lineal: varias líneas avanzaron en paralelo y algunas se pausaron a propósito. La figura 2 resume cuándo se trabajó en cada una. Los días sin barras son días sin sesiones; la actividad se concentró en tres tramos (23-25 de octubre, 31 de octubre-1 de noviembre y 2-3 de noviembre).

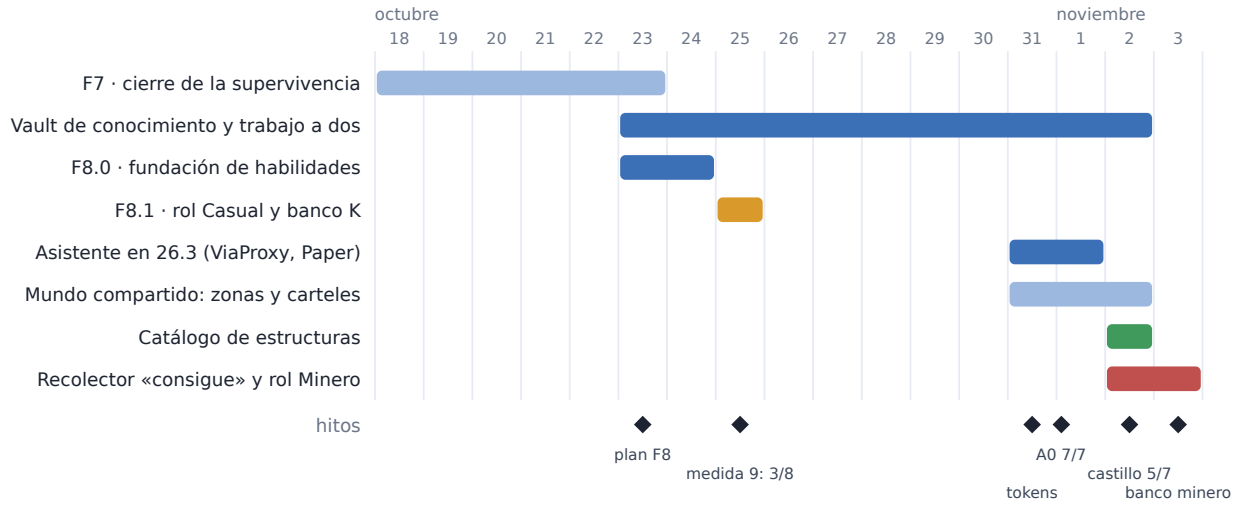


Figura 2. Diagrama de Gantt del periodo (18-10 a 03-11). Cada barra es una línea de trabajo; los rombos, hitos medidos.

Tramo	Qué se hizo	Por qué importa
18-23 oct	Cierre de la supervivencia (F7): la red de reflejos se entrena imitando al guion (clonación con DAgger). Se crea el **vault** de conocimiento y el plan de la fase 8.	Se pasa de «aprender a sobrevivir» a «aprender a jugar». El conocimiento deja de estar disperso.
24-25 oct	Fundación de habilidades nuevas (fundir, antorchas, escalera, cazar, cubo…) y primer rol completo, el **Casual**, medido con el banco K.	Primera prueba de un objetivo largo: «pico de hierro y cama en dos días».
31 oct-1 nov	Llegar a la versión nueva del juego (26.3) a través de un traductor de protocolo; movilidad en servidores Paper; puertas; zonas de trabajo marcadas con carteles; **economía de tokens**.	Los bots entran en el mundo real del usuario y del compañero, no solo en el de pruebas.
2-3 nov	**Catálogo de estructuras** (castillo en servidor creativo), **recolector** «consigue» y rol **Minero**, y el banco del recolector.	Los dos grandes propósitos del jugador: construir y obtener recursos raros.

3. De sobrevivir a jugar con un propósito

Un **rol** es una forma de jugar con un objetivo de largo horizonte. Se diseñaron siete: los principales (Casual, Speedrunner, Redstoner, Builder) y los de apoyo (Granjero, Minero, Explorador). Cada rol es una lista ordenada de **etapas**; cada etapa tiene un objetivo comprobable («tengo pico de hierro») y las acciones que lo consiguen. Los perfiles del informe anterior (talar más, explorar más…) pasan a ser *estilos* dentro de los roles.

3.1 Primero, cimientos medibles

Antes de juntar etapas, cada habilidad nueva se probó por separado en un servidor propio, con criterios numéricos. Por ejemplo, la habilidad de **explorar** llegó a su destino en llano 10/10, montaña 10/10, río 10/10 y cueva 8/10. Solo cuando las piezas pasan se monta el rol entero: si falla, se sabe que el fallo está en cómo se combinan, no en una pieza.

3.2 El banco K: medir un objetivo de dos días

El rol Casual debía conseguir **pico de hierro y cama en dos días de juego**. El **banco K** lanza 8 bots desde 8 puntos fijos del mundo, cuyo terreno se restaura idéntico antes de cada medida para que todas las medidas sean comparables.

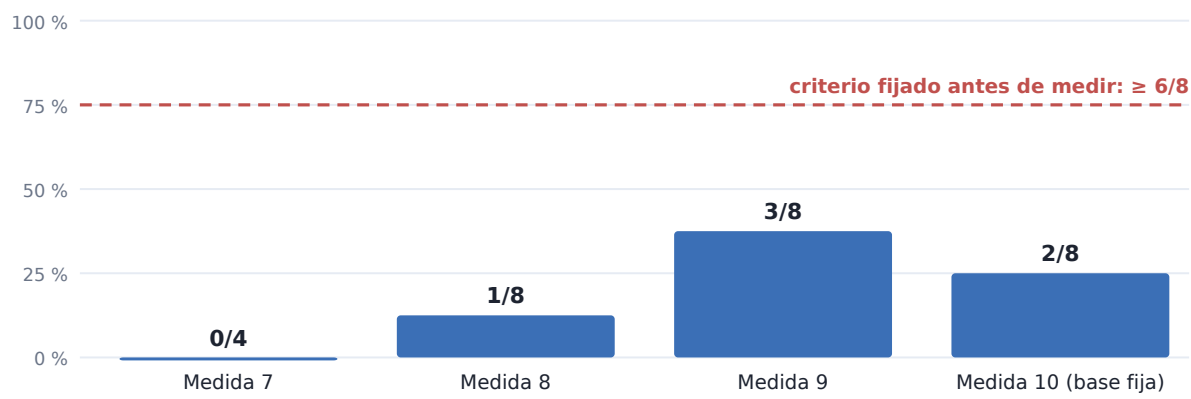


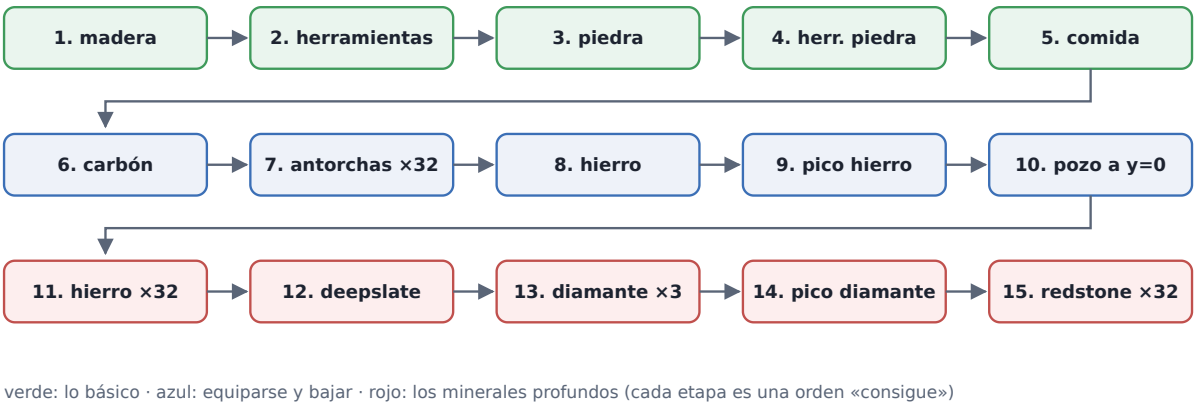
Figura 3. Resultados del banco K por medida. La línea roja es el criterio de aprobado (≥ 6 de 8), fijado antes de la primera medida. La medida 7 usó 4 bots.

El rol **no pasó todavía**. Pero la medida enseñó dónde está el problema: el pico de hierro ya sale 7/8 veces; lo que falla es la cama. Tras la etapa del hierro el bot está bajo tierra y la habilidad de cazar seguía a una oveja de la superficie *sin subir*: 19 de 28 cacerías acabaron así. Un fallo así no se ve probando a mano; aparece al repetir con método.

> **Ejemplo didáctico: ¿por qué fijar el criterio antes de medir?**
>
> Imagine que lanza un dado 8 veces buscando seises y salen 3. Si decide *después* que «3 está bien», siempre encontrará un motivo para aprobar. Fijando antes «necesito 6 de 8», el resultado no se puede negociar: 3/8 es un suspenso, y obliga a buscar la causa. En el proyecto, cada banco escribe su criterio en la nota del experimento antes de lanzarse.

4. El capataz: planificar sin gastar IA

El capataz recorre las etapas de un rol en orden. En cada momento mira el estado del bot (inventario, altura, vida, hambre) y elige **la primera etapa cuyo objetivo no está cumplido**. Es un algoritmo sencillo, determinista y gratuito: no llama a ninguna IA. Si una etapa falla, lo anota con su causa y lo reintenta o pide ayuda.



verde: lo básico · azul: equiparse y bajar · rojo: los minerales profundos (cada etapa es una orden «consigue»)

Figura 4. Las 15 etapas del rol Minero, en el orden en que el capataz las recorre. Las de la última fila son órdenes «consigue» (sección 5).

> **Ejemplo: cómo decide el capataz**
> Un Minero aparece con un pico de piedra y 3 panes. El capataz recorre: «¿madera?» ✓ (tiene herramientas, así que la etapa ya se dio), «¿herramientas de piedra?» ✓, «¿comida?» ✓, «¿carbón para 32 antorchas?» ✗ → **esa** es la etapa. Si entre tanto se le rompe el pico, en la siguiente vuelta la primera etapa sin cumplir será «herramientas de piedra» y volverá a ella sola, sin que nadie se lo diga.

5. El recolector: «Oqva, consigue 5 diamantes»

El personaje principal (Oqva, antes Soma) recibe una orden en lenguaje natural por el chat. El reto no es picar el diamante, sino **todo lo que hay en medio**: un diamante solo se puede extraer con pico de hierro; el hierro hay que fundirlo; abajo está oscuro y hay lava. El recolector convierte la orden en un **árbol de submetas** y lo resuelve de abajo arriba.

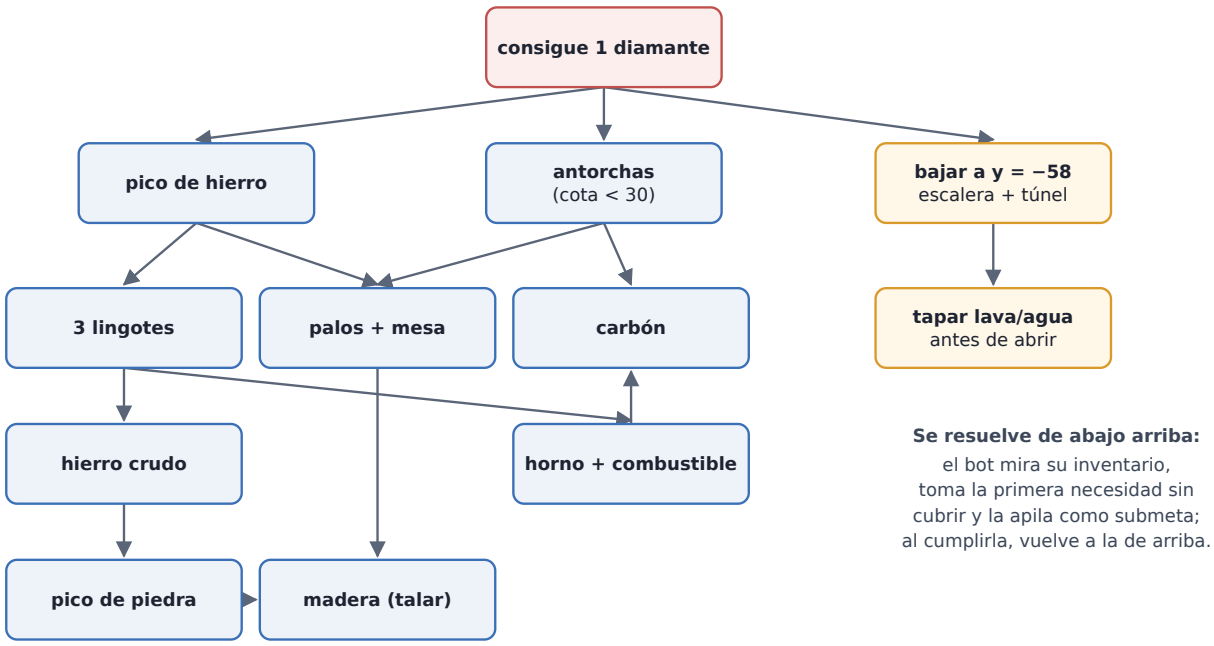


Figura 5. Árbol (simplificado) de submetas de «consigue 1 diamante». Cada flecha se lee «necesita». En naranja, los pasos de riesgo.

5.1 Dónde están los minerales

Cada mineral aparece sobre todo en una franja de altura. El recolector usa esa tabla para decidir hasta dónde bajar: no tiene sentido buscar diamantes a la altura del carbón.

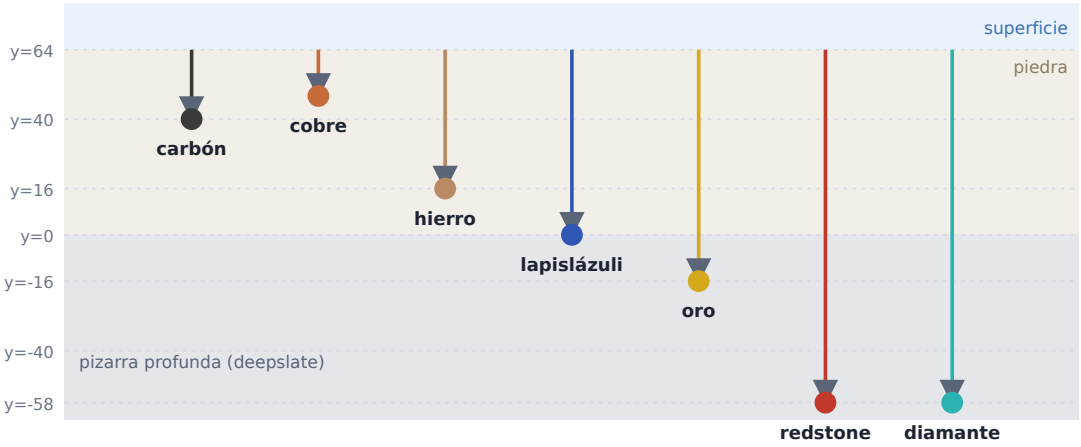
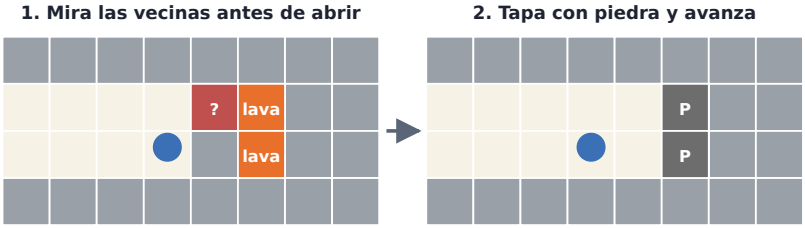


Figura 6. Altura a la que el recolector busca cada mineral (por debajo de y = 0 empieza la pizarra profunda; el mar y la superficie están hacia y = 63-70). La esmeralda solo sale en montañas y la netherite exige ir a otra dimensión (el Nether), así que quedan fuera de esta fase.

5.2 Seguridad: el túnel que tapa la lava

La causa de muerte más frecuente al minar es abrir una pared detrás de la cual hay lava o agua. El recolector cava túneles de 1×2 y, **antes de abrir** cada casilla, mira sus vecinas: si alguna es líquida, la tapa con piedra y solo entonces avanza. Cada 8 bloques pone una antorcha.



azul: el bot · crema: túnel abierto · gris: roca · naranja: lava · P: piedra puesta por el bot

Figura 7. Regla del túnel seguro, vista de perfil.

5.3 Resultados (provisionales)

Banco del recolector: un bot temporal aparece en un llano con un equipo inicial, recibe «consigue N material» y se cuenta lo que tiene al final. Cada intento empieza en otro punto, separado 48 bloques del anterior, para que el terreno no se repita. **Criterio fijado antes de medir:** éxito en ≥ 4 de 5 intentos por material y ninguna muerte en todo el banco.

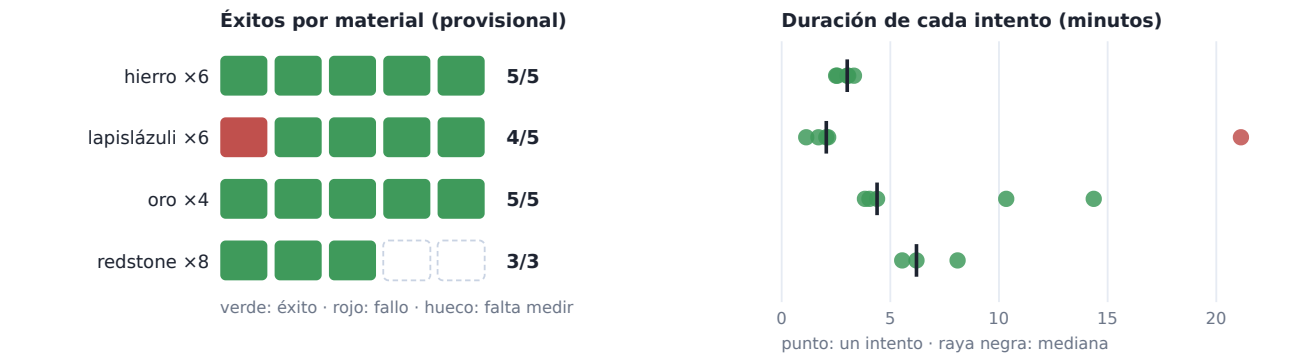


Figura 8. Izquierda: cada casilla es un intento. Derecha: duración de cada intento. El banco quedó pausado el 3-11 por decisión del usuario; faltan redstone (2), diamante (5) y diamante desde inventario vacío (5).

Material	Pide	Éxitos	mín. (min)	mediana	máx.	Criterio ≥ 4/5
Hierro	6	5/5	2.5	3.0	3.3	pasa
Lapislázuli	6	4/5	1.1	2.1	21.1	pasa
Oro	4	5/5	3.8	4.4	14.4	pasa
Redstone	8	3/3	5.5	6.2	8.1	pasa (faltan 2)
Diamante	2	—				sin medir
Diamante desde cero	1	—				sin medir

> **Lectura honesta**
> Lo medido pasa (5/5, 4/5, 5/5 y 3/3, con 0 muertes), pero **falta lo más difícil**: el diamante, a la máxima profundidad, y empezar desde el inventario vacío. El único fallo es instructivo: el lapislázuli exige pico de piedra, el pico se gastó a mitad de faena (5 de 6) y el bot, correctamente, se detuvo explicando la causa en lugar de seguir picando sin herramienta. Le faltó *reponer el pico por sí mismo*, que es justo lo que el árbol de submetas debería cubrir.

6. Construir: el catálogo de estructuras

Hasta ahora los bots solo levantaban planos hechos a mano y de un solo material. Ahora importan **plantillas reales** descargadas en los formatos habituales de la comunidad (.litematic, .schem, .nbt) y las convierten en una lista ordenada de bloques. Construir bien es más que poner bloques: hay que **orientar** escaleras y puertas, poner lo que cuelga *después* de su soporte, montar **andamio** cuando la capa queda alta y **abastecerse** en el cofre por tandas.



Figura 9. Ejemplo didáctico: una casita de tres capas y las reglas que sigue el bot al colocarla.

Obra pequeña: casita de prueba

99 de 99 bloques, tanto con la mochila llena como abasteciéndose de un cofre. La orden «deshaz» la retira.

Obra grande: torre de un castillo

Un castillo de 132.000 bloques no cabe en el mundo compartido, así que se montó un **servidor creativo** propio. La torre recortada: 1.086 de 3.210 bloques, **sin un solo error de orientación**; 5 de 7 pruebas pasan. Se detiene en la capa 9 por el andamio, el siguiente arreglo.

> **Lo que enseñó la obra grande**
>
> El navegador del bot trataba las casillas de la obra como «muy caras» y, por encima de cierto coste, **prohibidas**: el bot no daba un paso dentro de su propia obra. Es un buen ejemplo de un fallo que solo aparece al escalar: con una casita nunca ocurre.

7. Varios personajes con un modelo compartido

Objetivo: 2-3 personajes en el mismo mundo usando **la misma red**, con la misma tasa de éxito que uno solo. Las fases A-C (simulador con varios cuerpos, red de equipo, carga del servidor) se cerraron antes de este periodo; en la D, la prueba con un solo personaje pasa (95 %) y la carga del servidor es asumible. **En este periodo no hubo medida nueva**: queda medir solo, dúo y trío desde los mismos puntos. Mientras, el trabajo en el mundo compartido avanzó en lo social: cada bot tiene **dueño**, los bots de un jugador reconocen a sus **aliados**, y cada dueño marca sus **zonas** (hogar, obras, vedadas) con carteles en el propio mundo.

8. Trabajar a dos, cada uno con su agente

En el proyecto trabajan dos personas, cada una con su propio asistente de programación (Claude Code). El riesgo es que cada agente redescubra lo que el otro ya sabe, o que tomen decisiones incompatibles. La solución fue un **vault de conocimiento** en Obsidian, dentro del propio repositorio, con una nota por cosa y una plantilla por tipo.

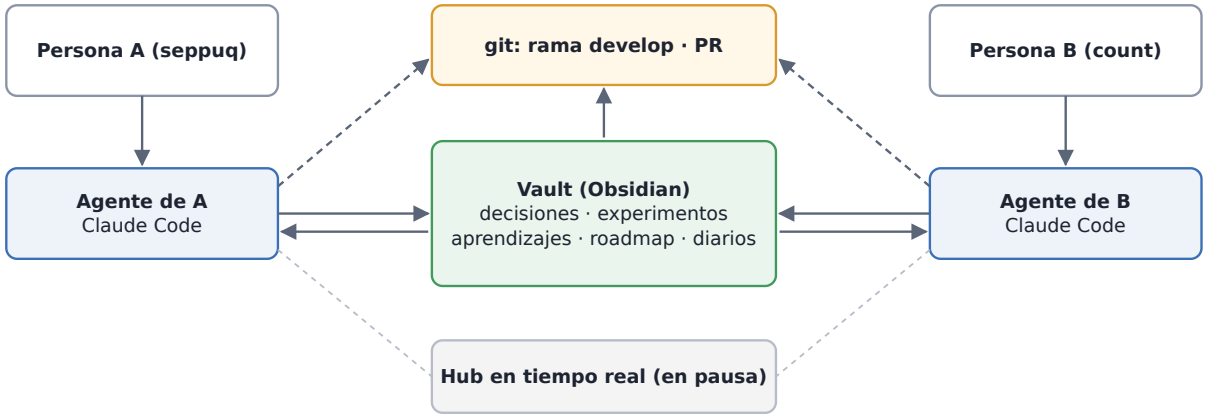


Figura 10. Flujo del conocimiento entre las dos personas. El vault viaja por git; el hub en tiempo real se probó y quedó en pausa el 2-11 porque no respondía.

Tipo de nota	Cuándo se escribe	Ejemplo real del periodo
Decisión	Algo que afecta al otro	«La madera de las plantillas es orientativa» (el bot taló abedul y la casita pedía roble).
Experimento	Termina una medida	Banco K, medida 9: 3/8, veredicto «mejora».
Aprendizaje	Un fallo con causa que no conviene redescubrir	«Un nido de abejas con abejas en un cofre desconecta al bot».
Roadmap	Cambia el estado o el siguiente paso	F8: «siguiente: ronda corta del Casual y otra medida del banco K».

Una regla sencilla resultó clave: **no se reescriben las decisiones**; si cambian, se escribe otra y la vieja se marca «sustituida». Así queda la historia de *por qué* se cambió de opinión (2 de las 30 decisiones están marcadas como sustituidas).

9. El coste como variable del experimento

El 31 de octubre, un solo día de trabajo consumió el 8 % del cupo semanal de IA. Se decidió entonces una **economía de tokens**: la máquina del usuario hace el bucle largo (servidores, bots, entrenamiento, bancos de horas) y la IA solo diseña, escribe código y **lee veredictos de una línea**, nunca registros enteros. A la línea del asistente se le puso un tope explícito del 5 % semanal.

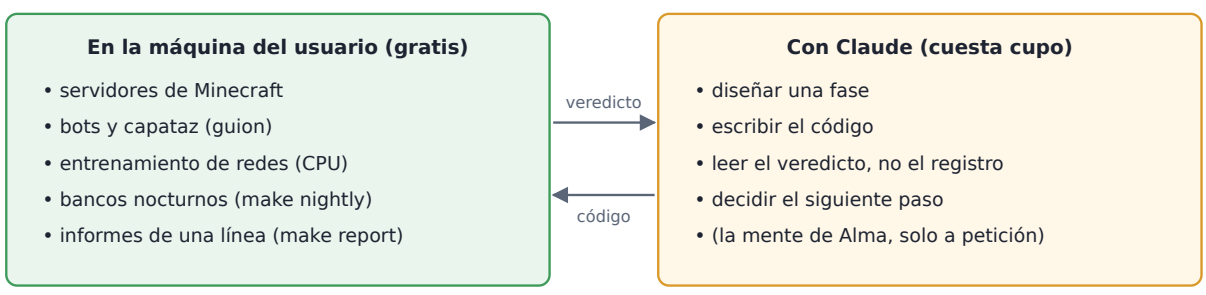


Figura 11. Reparto del trabajo entre la máquina local y la IA.

> **Ejemplo: cuánto cuesta «mirar»**
> Un banco de 30 intentos genera miles de líneas de registro. Si la IA las leyera para saber cómo va, gastaría más que escribiendo el código del banco. En cambio, el banco escribe un JSON con una fila por intento y una línea final («hierro 5/5 PASA · muertes 0»). La IA lee esa línea. Es la misma idea que un resumen ejecutivo: el detalle existe, pero solo se abre si el resumen dice que algo va mal.

El cuerpo de los bots tampoco necesita IA grande: la red de reflejos es pequeña (con estas redes, el procesador normal resultó más rápido que la tarjeta gráfica) y el capataz es guion. La IA grande queda como un recurso escaso que se usa con intención.

10. Lo que falló, y lo que enseñó

Qué pasó	Causa	Qué se cambió
El primer bot del recolector murió ahogado.	El punto de salida cayó en un lago y el bot no sabía salir del agua honda.	Salir del agua es lo primero; el banco descarta puntos con agua.
Un reinicio del PC borró horas de banco.	El resultado solo se escribía al final.	Se guarda tras cada intento y se puede reanudar sin repetir.
Una pausa contó como fallo.	El intento cortado a mano se guardó como «cancelado».	Un intento cortado no se cuenta; se repite al reanudar.
El bot no avanzaba dentro de su obra.	Un coste del navegador por encima de 100 equivale a «prohibido».	Coste menor en la huella de las obras grandes.
Los bots no cruzaban puertas.	El navegador trataba la puerta como un muro.	Las puertas de madera cuentan como paso; el bot abre y cierra detrás.
La cama del Casual no salía.	Cazar seguía a la oveja de la superficie desde la mina.	Cazar sube primero a la superficie (pendiente de medir).
El servidor nuevo (26.3) expulsaba al bot.	La versión nueva exige un mensaje por tick que la librería no enviaba.	Un envoltorio propio que lo envía, a través de un traductor de protocolo (puerta A0: 7/7).

Patrón común: casi ningún fallo era del «cerebro» del bot; la mayoría eran **fricciones con el mundo** (agua, puertas, versiones, el propio montaje de la prueba). Es coherente con lo visto en el informe 2: el cuello de botella está en la interfaz entre el agente y su entorno.

11. Dónde estamos y qué falta

	Línea	Estado	Siguiente paso
🟢	Habilidades base (F8.0)	Hechas y medidas	—
🟢	Recolector «consigue»	Provisional: pasa lo medido, 0 muertes	Terminar el banco (diamante y desde cero); reponer la herramienta gastada.
🟡	Rol Minero	Diseñado y escrito; sin medir	Banco de dos días del Minero.
🔴	Rol Casual (banco K)	3/8 (criterio 6/8)	Ronda corta de arreglos y otra medida.
🟡	Catálogo de estructuras	Casita 99/99; torre 5/7	Andamio desde la capa 9; castillo entero.
🟡	Varios personajes	Fase D a medias	Medir solo, dúo y trío.
🟡	Asistente en el mundo del usuario	Puerta A0 7/7; en pausa por cupo	A1 desde el 7-11.
🟡	Mente (Alma)	Código escrito; sin medida formal	Medir T1-T3 (solo a petición: consume IA).
⚪	Hub en tiempo real	En pausa	Reactivar si vuelve a hacer falta.

> **Conclusión del periodo**
> El proyecto ha pasado de demostrar que un agente *sobrevive* a demostrar que puede *perseguir un encargo* de varios pasos con riesgo real (bajar a la mina, esquivar lava, volver con lo pedido). La arquitectura en tres capas permite hacerlo sin que cada paso cueste IA. Lo que queda abierto es la robustez a horizontes largos (dos días de juego) y la coordinación entre varios personajes.

12. Pequeño glosario

Banco
Prueba repetible con condiciones controladas (mismo terreno, mismo equipo) y un criterio de aprobado fijado de antemano.

Capataz
Planificador escrito a mano que elige la siguiente etapa de un rol según el estado del bot. No usa IA.

Clonación con DAgger
Técnica para entrenar una red imitando a un experto (aquí, el guion) y corrigiéndola en las situaciones donde se equivoca.

Etapa
Paso de un rol con un objetivo comprobable («tengo pico de hierro») y las acciones que lo consiguen.

Prueba en seco

Prueba del código con un mundo simulado, sin servidor: rápida y gratuita. Se contrapone a la prueba «en el juego».

Rol
Forma de jugar con un objetivo largo (Casual, Minero, Builder…), compuesta por etapas.

Submeta
Requisito intermedio que hay que cumplir antes de la meta (el pico antes del diamante).

Vault
Carpeta de notas enlazadas (Obsidian) que guarda decisiones, experimentos y aprendizajes del proyecto, compartida por git.

Fuentes: vault `conocimiento/` (roadmap, 30 decisiones, experimentos, 24 aprendizajes, diarios), historial de git desde el 18-10-2026 y `servidor-supervivencia/logs/conseguir-20261102-2257.json` (banco del recolector, provisional). Informes anteriores: `resumen/INFORME_Experimento_A_2026-10-15.pdf` y `resumen/INFORME_Supervivencia_2026-10-18.pdf`.